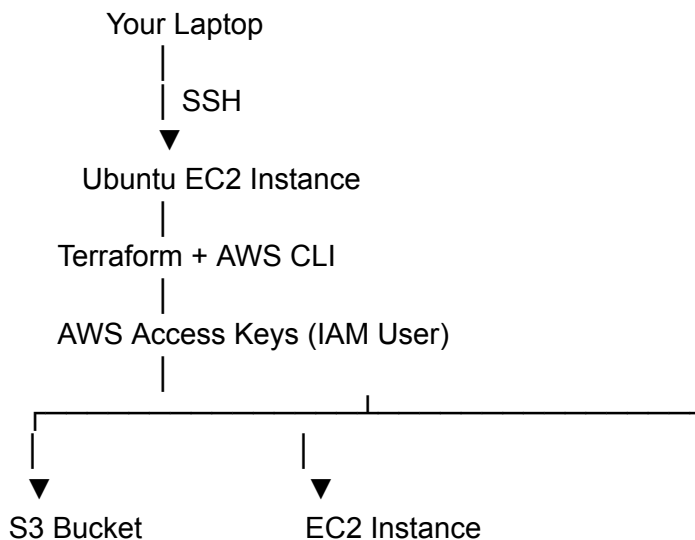


Day 61 – Introduction to Terraform and Your First AWS Infrastructure

Architecture



Step 1: Launch an EC2 Instance

If you already have one, use it.

Recommended:

- Ubuntu 22.04
- t2.medium (recommended for your learning journey)
- 20 GB storage
- Security Group
 - SSH (22) from your IP

SSH into it

```
ssh -i your-key.pem ubuntu@<Public-IP>
```

Update packages

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install AWS CLI

Check if installed

```
aws --version
```

If not,

```
sudo apt install awscli -y
```

Verify

```
aws --version
```

```
ubuntu@ip-172-31-82-160:~$ aws --version
aws-cli/2.31.35 Python/3.14.4 Linux/7.0.0-1006-aws source/x86_64.ubuntu.26
ubuntu@ip-172-31-82-160:~$
```

Step 3: Create an IAM User

Do **NOT** use the root account.

AWS Console

IAM

↓

Users

↓

Create User

Name

User-A

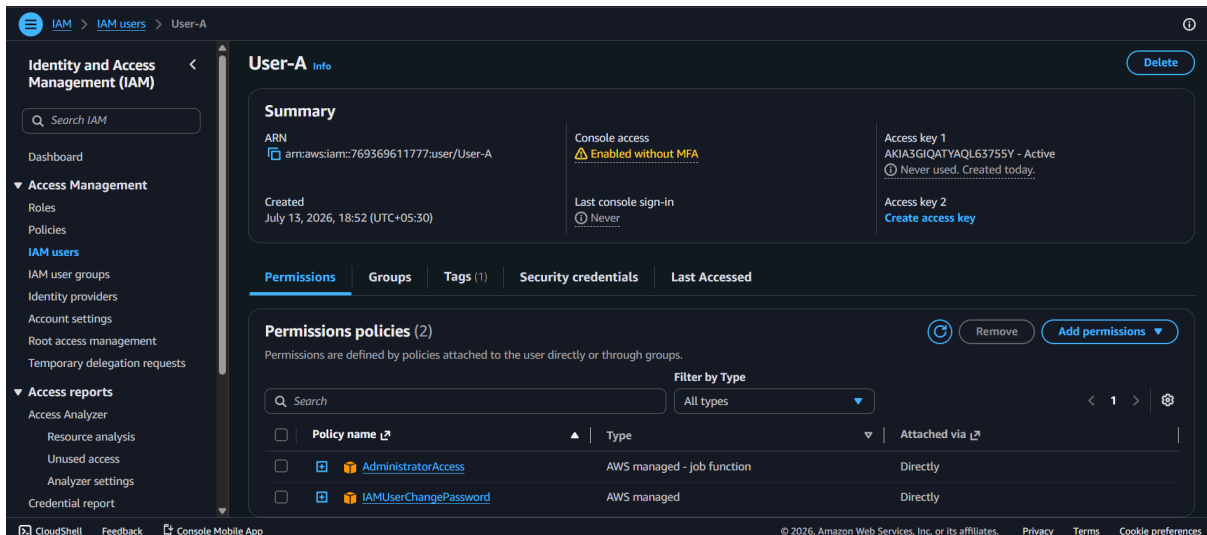
Next

Attach policies

For learning

AdministratorAccess

Create User.



Create Access Key

Open

User-A



Security Credentials



Create Access Key



Command Line Interface (CLI)



Create

Download the credentials.

You'll get

Access Key ID

Secret Access Key

Keep these safe.

Step 4: Configure AWS CLI

Run

aws configure

Enter

AWS Access Key ID:

Paste key

AWS Secret Access Key:

Paste secret

Default region:

Example

ap-south-1

Output

json

```
ubuntu@ip-172-31-79-174:~$ aws configure
AWS Access Key ID [None]: AKIA3GIQATYAQL63755Y
AWS Secret Access Key [None]: msy-awb-3cm8cc38-1m3-jt-55u0113-1m3-jm3-awb
Default region name [None]: us-east-1
Default output format [None]: json
ubuntu@ip-172-31-79-174:~$
```

Step 5: Verify AWS Connection

Run

aws sts get-caller-identity

Expected

```
{
  "UserId": "...",
  "Account": "123456789",
  "Arn": "arn:aws:iam::123456789:user/terraform-user"
}
```

Congratulations!

```
ubuntu@ip-172-31-79-174:~$ aws sts get-caller-identity
{
  "UserId": "AIDA3GIQATYAQZXAORJKE",
  "Account": "769369611777",
  "Arn": "arn:aws:iam::769369611777:user/User-A"
}
ubuntu@ip-172-31-79-174:~$
```

Terraform can now communicate with AWS.

Step 6: Install Terraform

Refer official doc to install terraform

<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>

Run

```
wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
```

```
ubuntu@ip-172-31-82-160:~$ wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
--2026-07-13 13:43:58-- https://apt.releases.hashicorp.com/gpg
Resolving apt.releases.hashicorp.com (apt.releases.hashicorp.com)... 18.160.10.126, 18.160.10.45, 18.160.10.69, ...
Connecting to apt.releases.hashicorp.com (apt.releases.hashicorp.com)|18.160.10.126|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3980 (3.9K) [binary/octet-stream]
Saving to: 'STDOUT'

-
100%[=====] 3.89K --KB/s in 0s
2026-07-13 13:43:58 (237 MB/s) - written to stdout [3980/3980]
```

Add repository

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com \
$(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list
```

```
ubuntu@ip-172-31-82-160:~$ echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com \
$(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list
deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main
```

Update

```
sudo apt update
```

Install

```
sudo apt install terraform -y
```

```
ubuntu@ip-172-31-82-160:~$ sudo apt install terraform -y
Installing:
  terraform

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 6
  Download size: 34.8 MB
  Space needed: 117 MB / 4123 MB available

Get:1 https://apt.releases.hashicorp.com resolute/main amd64 terraform amd64 1.15.8-1 [34.8 MB]
Fetched 34.8 MB in 0s (104 MB/s)
Selecting previously unselected package terraform.
(Reading database ... 133892 files and directories currently installed.)
Preparing to unpack .../terraform_1.15.8-1_amd64.deb ...
Unpacking terraform (1.15.8-1) ...
Setting up terraform (1.15.8-1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...
```

Verify

terraform -version

```
ubuntu@ip-172-31-79-174:~$ terraform -version
Terraform v1.15.8
on linux_amd64
ubuntu@ip-172-31-79-174:~$
```

Step 7: Create Project Folder

mkdir terraform-basics

Go inside

cd terraform-basics

Check

pwd

Here's why each step is in that order:

1. **terraform init**
 - Sets up the working directory.
 - Downloads providers and modules.
 - Must be run before **validate** or **plan**.
2. **terraform fmt**
 - Formats your Terraform code.
 - Makes it easier to read and review.
3. **terraform validate**
 - Ensures the configuration is syntactically and structurally correct.

- Catches errors early before planning.
 - 4. **terraform plan**
 - Shows exactly what Terraform intends to do (create, modify, or destroy resources).
 - Gives you a chance to review the changes.
 - 5. **terraform apply**
 - Applies the reviewed changes to your infrastructure.
-

Step 8: Create Files

touch main.tf

Open

nano main.tf

Paste

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

```
provider "aws" {
  region = "us-east-1"
}
```

```
resource "aws_s3_bucket" "mybucket" {

  bucket = "terraform-2026-demo"

}
```

⚠ Bucket names must be **globally unique**. Change **parthavi-terraform-2026-demo** if it is already taken.

Save

CTRL + O
ENTER

CTRL + X

Step 9: Format Terraform Code

Run

terraform fmt

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform fmt
main.tf
ubuntu@ip-172-31-79-174:~/terraform-basics$
```

Step 10: Validate Syntax

terraform validate

Expected

Success!

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform validate
Success! The configuration is valid.

ubuntu@ip-172-31-79-174:~/terraform-basics$
```

Step 11: Initialize Terraform

Run

terraform init

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 6.0"...
- Installing hashicorp/aws v6.55.0...
- Installed hashicorp/aws v6.55.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-79-174:~/terraform-basics$
```


This downloads:

- AWS Provider plugin
- Provider dependencies
- Creates `.terraform/`
- Creates `.terraform.lock.hcl`

Example output

Initializing...

Downloading AWS Provider...

Terraform has been successfully initialized!

Step 12: Check Files

ls -la

You'll see

main.tf

.terraform/

.terraform.lock.hcl

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ ls -la
.   ..  .terraform  .terraform.lock.hcl  main.tf
ubuntu@ip-172-31-79-174:~/terraform-basics$
```

Step 13: Plan

Run

terraform plan

```

ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.mybucket will be created
+ resource "aws_s3_bucket" "mybucket" {
+   acceleration_status      = (known after apply)
+   acl                     = (known after apply)
+   arn                    = (known after apply)
+   bucket                 = "terraform-2026-demo"
+   bucket_domain_name     = (known after apply)
+   bucket_namespace       = (known after apply)
+   bucket_prefix          = (known after apply)
+   bucket_region          = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy          = false
+   hosted_zone_id         = (known after apply)
+   id                     = (known after apply)
+   object_lock_enabled     = (known after apply)
+   policy                 = (known after apply)
+   region                 = "us-east-1"
+   request_payer          = (known after apply)
+   tags_all               = (known after apply)
+   website_domain         = (known after apply)
+   website_endpoint       = (known after apply)

+   cors_rule (known after apply)

+   grant (known after apply)

+   lifecycle_rule (known after apply)

+   server_side_encryption_configuration (known after apply)

+   versioning (known after apply)

+   website (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

```

Terraform compares:

Current State

VS

Desired State

Expected

Plan:

1 to add

0 to change

0 to destroy

Nothing is created yet.

Step 14: Apply

Run

terraform apply

Type

yes

Terraform creates

S3 Bucket

Go to AWS Console

↓

S3

↓

Refresh

You should see your bucket.

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_s3_bucket.mybucket will be created
+ resource "aws_s3_bucket" "mybucket" {
+   acceleration_status = (known after apply)
+   acl                  = (known after apply)
+   arn                  = (known after apply)
+   bucket               = "terraform-2026-demo"
+   bucket_domain_name  = (known after apply)
+   bucket_namespace    = (known after apply)
+   bucket_prefix       = (known after apply)
+   bucket_region       = (known after apply)
+   bucket_regional_domain_name = (known after apply)
+   force_destroy       = false
+   hosted_zone_id      = (known after apply)
+   id                   = (known after apply)
+   object_lock_enabled = (known after apply)
+   policy               = (known after apply)
+   region               = "us-east-1"
+   request_payer       = (known after apply)
+   tags_all             = (known after apply)
+   website_domain      = (known after apply)
+   website_endpoint    = (known after apply)

+ cors_rule (known after apply)

+ grant (known after apply)

+ lifecycle_rule (known after apply)

+ logging (known after apply)
```

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_bucket.mybucket: Creating...
aws_s3_bucket.mybucket: Creation complete after 0s [id=terraform-2026-demo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```

Bucket created

Buckets

General purpose buckets **All AWS Regions** Directory buckets

General purpose buckets (5) [Info](#) [Copy ARN](#) [Empty](#) [Delete](#) [Create bucket](#)

Buckets are containers for data stored in S3.

	Name	AWS Region	Creation date
☐	aws-athena-query-results-ap-south-1-769369611777	Asia Pacific (Mumbai) ap-south-1	March 11, 2026, 18:46:45 (UTC+05:30)
☐	aws-cloudtrail-logs-769369611777-500ee5e2	Asia Pacific (Mumbai) ap-south-1	February 4, 2026, 12:50:59 (UTC+05:30)
☐	cur-report-si	Asia Pacific (Mumbai) ap-south-1	March 13, 2026, 15:33:33 (UTC+05:30)
☐	quicksight-cur-si	US East (N. Virginia) us-east-1	March 11, 2026, 11:41:17 (UTC+05:30)
☐	terraform-2026-demo	US East (N. Virginia) us-east-1	July 16, 2026, 09:09:35 (UTC+05:30)

Step 15: Add EC2

Open

nano main.tf

Add

```

resource "aws_instance" "web" {

  ami          = "ami-0f5ee92e2d63afc18"

  instance_type = "t2.micro"

  tags = {

```

```
    Name = "TerraWeek-Day1"
  }
}
```

Save.

Run

terraform fmt

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform fmt
main.tf
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform validate
Success! The configuration is valid.
```

Plan

terraform plan

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform plan
aws_s3_bucket.mybucket: Refreshing state... [id=terraform-2026-demo]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-0b6d9d3d33ba97d99"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + enable_primary_ipv6    = (known after apply)
  + force_destroy         = false
  + get_password_data      = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle     = (known after apply)
  + instance_state         = (known after apply)
  + instance_type          = "t2.micro"
  + ipv6_address_count     = (known after apply)
  + ipv6_addresses         = (known after apply)
  + key_name               = (known after apply)
  + monitoring              = (known after apply)
  + outpost_arn            = (known after apply)
  + password_data          = (known after apply)
  + placement_group        = (known after apply)
  + network_interface       = (known after apply)
  + primary_network_interface = (known after apply)
  + private_dns_name_options = (known after apply)
  + root_block_device       = (known after apply)
  + secondary_network_interface = (known after apply)
}
```

```
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Expected
1 to add

0 to change

0 destroy

Notice the S3 bucket is **not** recreated because Terraform knows it already exists (it records it in the state file).

Apply

terraform apply

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform apply
aws_s3_bucket.mybucket: Refreshing state... [id=terraform-2026-demo]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-0b6d9d3d33ba97d99"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + disable_api_stop     = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + enable_primary_ipv6   = (known after apply)
  + force_destroy         = false
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile  = (known after apply)
  + id                    = (known after apply)
}
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.
```

```
Enter a value: yes
```

```
aws_instance.web: Creating...
aws_instance.web: Still creating... [00m10s elapsed]
aws_instance.web: Creation complete after 13s [id=i-0b27f14fa0a2d5517]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Type

yes

Go to

AWS Console

↓

EC2

↓

Instances

You'll see

TerraWeek-Day1

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐	web-server	i-0d7b5d8081ea6e103	Running	t2.medium	2/2 checks passed	us-east-1f	ec2-100-54-65-108.co...	100.54.65.108	-
☐	TerraWeek-Day1	i-0b27f14fa0a2d5517	Running	t2.micro	Initializing	us-east-1c	ec2-34-224-169-116.co...	34.224.169.116	-

Running

Step 16: Inspect State

List files

ls

You'll see

terraform.tfstate

Open it

cat terraform.tfstate

You'll see JSON containing IDs, ARNs, tags, region, resource names, and more.

```

},
{
  "mode": "managed",
  "type": "aws_s3_bucket",
  "name": "mybucket",
  "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
  "instances": [
    {
      "schema_version": 0,
      "attributes": {
        "acceleration_status": "",
        "acl": null,
        "arn": "arn:aws:s3:::terraform-2026-demo",
        "bucket": "terraform-2026-demo",
        "bucket_domain_name": "terraform-2026-demo.s3.amazonaws.com",
        "bucket_namespace": "global",
        "bucket_prefix": "",
        "bucket_region": "ap-south-1",
        "bucket_regional_domain_name": "terraform-2026-demo.s3.ap-south-1.amazonaws.com",
        "cors_rule": [],
        "force_destroy": false,
        "grant": [
          {
            "id": "cb2b3191f179191ab571908d73590e9edd9c394910781cc96bca72508d23f5de",
            "permissions": [
              "FULL_CONTROL"
            ],
            "type": "CanonicalUser",
            "uri": ""
          }
        ],
        "hosted_zone_id": "Z11RGJOFQNVJUP",
        "id": "terraform-2026-demo",
        "lifecycle_rule": [],
        "logging": [],
        "object_lock_configuration": [],
        "object_lock_enabled": false,
        "policy": "",
        "region": "ap-south-1",
        "replication_configuration": [],

```

Step 17: Terraform Show

terraform show


```

ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform show
# aws_instance.web:
resource "aws_instance" "web" {
  ami                    = "ami-0b6d9d3d323ba97d99"
  arn                   = "arn:aws:ec2:us-east-1:769369611777:instance/i-0b27f14fa0a2d5517"
  associate_public_ip_address = true
  availability_zone      = "us-east-1c"
  disable_api_stop       = false
  disable_api_termination = false
  cbs_optimized          = false
  force_destroy          = false
  get_password_data      = false
  hibernation             = false
  host_id                = null
  iam_instance_profile    = null
  id                     = "i-0b27f14fa0a2d5517"
  instance_initiated_shutdown_behavior = "stop"
  instance_lifecycle      = null
  instance_state          = "running"
  instance_type           = "t2.micro"
  ipv6_address_count      = 0
  ipv6_addresses          = []
  key_name                = null
  monitoring              = false
  outpost_arn             = null
  password_data           = null
  placement_group         = null
  placement_group_id      = null
  placement_partition_number = 0
  primary_network_interface_id = "eni-02a5631b6723f8bb0"
  private_dns             = "ip-172-31-29-132.ec2.internal"
  private_ip              = "172.31.29.132"
  public_dns              = "ec2-34-224-169-116.compute-1.amazonaws.com"
  public_ip               = "34.224.169.116"
  region                  = "us-east-1"
  secondary_private_ips   = []
  security_groups         = [
    "default",
  ]
  source_dest_check       = true
  spot_instance_request_id = null
  subnet_id               = "subnet-02908a6bfaf0a652c"
  tags                    = {
    "Name" = "TerraWeek-Day1"
  }
  tags_all                = {
    "Name" = "TerraWeek-Day1"
  }
  tenancy                  = "default"
}

```

```
# aws_s3_bucket.mybucket:
resource "aws_s3_bucket" "mybucket" {
  acceleration_status      = null
  arn                      = "arn:aws:s3:::terraform-2026-demo"
  bucket                  = "terraform-2026-demo"
  bucket_domain_name      = "terraform-2026-demo.s3.amazonaws.com"
  bucket_namespace       = "global"
  bucket_prefix           = null
  bucket_region          = "us-east-1"
  bucket_regional_domain_name = "terraform-2026-demo.s3.us-east-1.amazonaws.com"
  force_destroy           = false
  hosted_zone_id          = "Z3AQBSTGPFYJSTF"
  id                      = "terraform-2026-demo"
  object_lock_enabled     = false
  policy                  = null
  region                  = "us-east-1"
  request_payer           = "BucketOwner"
  tags                    = {}
  tags_all                = {}

  grant {
    id          = "cb2b3191f179191ab571908d73590e9edd9c394910781cc96bca72508d23f5de"
    permissions = [
      "FULL_CONTROL",
    ]
    type        = "CanonicalUser"
    uri         = null
  }

  server_side_encryption_configuration {
    rule {
      bucket_key_enabled = false

      apply_server_side_encryption_by_default {
        kms_master_key_id = null
        sse_algorithm     = "AES256"
      }
    }
  }

  versioning {
    enabled      = false
    mfa_delete = false
  }
}
```

Shows the current infrastructure in a human-readable format.

Step 18: State List

terraform state list

Expected

aws_instance.web

Aws_s3_bucket.mybucket

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform state list
aws_instance.web
aws_s3_bucket.mybucket
```

Step 19: State Show

S3

terraform state show aws_s3_bucket.mybucket

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform state show aws_s3_bucket.mybucket
# aws_s3_bucket.mybucket:
resource "aws_s3_bucket" "mybucket" {
  acceleration_status      = null
  arn                      = "arn:aws:s3:::terraform-2026-demo"
  bucket                   = "terraform-2026-demo"
  bucket_domain_name      = "terraform-2026-demo.s3.amazonaws.com"
  bucket_namespace        = "global"
  bucket_prefix            = null
  bucket_region           = "us-east-1"
  bucket_regional_domain_name = "terraform-2026-demo.s3.us-east-1.amazonaws.com"
  force_destroy            = false
  hosted_zone_id          = "23AQBSTGIFYJSTF"
  id                      = "terraform-2026-demo"
  object_lock_enabled      = false
  policy                   = null
  region                   = "us-east-1"
  request_payer            = "BucketOwner"
  tags                     = {}
  tags_all                 = {}

  grant {
    id      = "cb2b3191f179191ab571908d73590e9edd9c394910781cc96bca72508d23f5de"
    permissions = [
      "FULL_CONTROL",
    ]
    type      = "CanonicalUser"
    uri      = null
  }

  server_side_encryption_configuration {
    rule {
      bucket_key_enabled = false

      apply_server_side_encryption_by_default {
        kms_master_key_id = null
        sse_algorithm     = "AES256"
      }
    }
  }

  versioning {
    enabled      = false
    mfa_delete   = false
  }
}
```

EC2

terraform state show aws_instance.web

```

ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform state show aws_instance.web
# aws_instance.web:
resource "aws_instance" "web" {
  ami                  = "ami-0b6d9d3d33ba97d99"
  arn                  = "arn:aws:ec2:us-east-1:769369611777:instance/i-0b27f14fa0a2d5517"
  associate_public_ip_address = true
  availability_zone     = "us-east-1c"
  disable_api_stop      = false
  disable_api_termination = false
  ebs_optimized         = false
  force_destroy         = false
  get_password_data     = false
  hibernation           = false
  host_id               = null
  iam_instance_profile  = null
  id                    = "i-0b27f14fa0a2d5517"
  instance_initiated_shutdown_behavior = "stop"
  instance_lifecycle    = null
  instance_state        = "running"
  instance_type         = "t2.micro"
  ipv6_address_count    = 0
  ipv6_addresses        = []
  key_name              = null
  monitoring            = false
  outpost_arn           = null
  password_data         = null
  placement_group       = null
  placement_group_id    = null
  placement_partition_number = 0
  primary_network_interface_id = "eni-02a5631b6723f8bb0"
  private_dns           = "ip-172-31-29-132.ec2.internal"
  private_ip            = "172.31.29.132"
  public_dns            = "ec2-34-224-169-116.compute-1.amazonaws.com"
  public_ip             = "34.224.169.116"
  region               = "us-east-1"
  secondary_private_ips = []
  security_groups       = [
    "default",
  ]
  source_dest_check     = true
  spot_instance_request_id = null
  subnet_id            = "subnet-02908a6bfaf0a652c"
  tags                 = {
    "Name" = "TerraWeek-Day1"
  }
  tags_all             = {
    "Name" = "TerraWeek-Day1"
  }
  tenancy               = "default"
}

```

This displays detailed information about each managed resource.

Step 20: Modify Tag

Open

nano main.tf

Change

Name = "TerraWeek-Day1"

to

Name = "TerraWeek-Modified"

Save.

Plan

terraform plan

You'll see

~ update in-place

Symbols:

- + = Create a new resource.
- ~ = Modify an existing resource.
- - = Destroy a resource.

Changing only a tag is an **in-place update**, so Terraform updates the instance without recreating it.

```
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform plan
aws_s3_bucket.mybucket: Refreshing state... [id=terraform-2026-demo]
aws_instance.web: Refreshing state... [id=i-0b27f14fa0a2d5517]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.web will be updated in-place
  ~ resource "aws_instance" "web" {
    id           = "i-0b27f14fa0a2d5517"
    ~ tags       = {
      ~ "Name" = "TerraWeek-Day1" -> "TerraWeek-Day1-modified"
    }
    ~ tags_all   = {
      ~ "Name" = "TerraWeek-Day1" -> "TerraWeek-Day1-modified"
    }
    # (39 unchanged attributes hidden)

    # (9 unchanged blocks hidden)
  }

Plan: 0 to add, 1 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
ubuntu@ip-172-31-79-174:~/terraform-basics$
```

Apply

terraform apply

```

ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform apply
terraform: command not found
ubuntu@ip-172-31-79-174:~/terraform-basics$ terraform apply
aws_s3_bucket.mybucket: Refreshing state... [id=terraform-2026-demo]
aws_instance.web: Refreshing state... [id=i-0b27f14fa0a2d5517]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

# aws_instance.web will be updated in-place
~ resource "aws_instance" "web" {
  id              = "i-0b27f14fa0a2d5517"
  ~ tags          = {
    ~ "Name" = "TerraWeek-Day1" -> "TerraWeek-Day1-modified"
  }
  ~ tags_all      = {
    ~ "Name" = "TerraWeek-Day1" -> "TerraWeek-Day1-modified"
  }
  # (39 unchanged attributes hidden)
  # (9 unchanged blocks hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

```

```

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.web: Modifying... [id=i-0b27f14fa0a2d5517]
aws_instance.web: Modifications complete after 1s [id=i-0b27f14fa0a2d5517]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

```

Verify in the EC2 console that the tag changed to **TerraWeek-Modified**.

Step 21: Destroy Everything

Run

terraform destroy

```

# aws_instance.web will be destroyed
- resource "aws_instance" "web" {
  ami              = "ami-0b6d9d3d33ba97d99" -> null
  arn              = "arn:aws:ec2:us-east-1:769369611777:instance/i-0b27f14fa0a2d5517" -> null
  associate_public_ip_address = true -> null
  availability_zone = "us-east-1c" -> null
  disable_api_stop    = false -> null
  disable_api_termination = false -> null
  ebs_optimized        = false -> null
  force_destroy        = false -> null
  get_password_data     = false -> null
  hibernation          = false -> null
}

```

```
# aws_s3_bucket.mybucket will be destroyed
- resource "aws_s3_bucket" "mybucket" {
  - arn                = "arn:aws:s3:::terraform-2026-demo" -> null
  - bucket             = "terraform-2026-demo" -> null
  - bucket_domain_name = "terraform-2026-demo.s3.amazonaws.com" -> null
  - bucket_namespace   = "global" -> null
  - bucket_region      = "us-east-1" -> null
  - bucket_regional_domain_name = "terraform-2026-demo.s3.us-east-1.amazonaws.com" -> null
  - force_destroy      = false -> null
  - hosted_zone_id     = "Z3AQBSTGFYJSTF" -> null
  - id                 = "terraform-2026-demo" -> null
  - object_lock_enabled = false -> null
  - region             = "us-east-1" -> null
  - request_payer      = "BucketOwner" -> null
}
```

```
aws_s3_bucket.mybucket: Destroying... [id=terraform-2026-demo]
aws_instance.web: Destroying... [id=i-0b27f14fa0a2d5517]
aws_s3_bucket.mybucket: Destruction complete after 1s
aws_instance.web: Still destroying... [id=i-0b27f14fa0a2d5517, 00m10s elapsed]
aws_instance.web: Still destroying... [id=i-0b27f14fa0a2d5517, 00m20s elapsed]
aws_instance.web: Destruction complete after 30s

Destroy complete! Resources: 2 destroyed.
```

Type

yes

Terraform deletes the EC2 instance and the S3 bucket.

Verify in the AWS Console that both resources are gone.

Step 22: Create **.gitignore**

nano .gitignore

Add

.terraform/

*.tfstate

*.tfstate.backup

Save the file.

Commands Summary

Command	Purpose
<code>terraform init</code>	Downloads providers and initializes the project.
<code>terraform fmt</code>	Formats Terraform files.
<code>terraform validate</code>	Checks syntax and configuration.
<code>terraform plan</code>	Shows what Terraform will do without making changes.
<code>terraform apply</code>	Creates or updates infrastructure.
<code>terraform show</code>	Displays the current state in a readable format.
<code>terraform state list</code>	Lists all resources managed by Terraform.
<code>terraform state show <resource></code>	Displays details of one resource.
<code>terraform destroy</code>	Deletes all managed infrastructure.
